

AI Interface Handbook

The bench-wide AI layer. Every app calls one contract; whichever provider is configured is the one that runs. Nothing about a vendor lives in code.

CONTENTS

- 01 What this is
 - 02 How a call flows
 - 03 First-time setup
 - 04 Adding any provider
 - 05 Using it from your app
 - 06 Doctype reference
 - 07 How routing decides
 - 08 Cost & currency
 - 09 Spend caps
 - 10 Provider health
 - 11 The dashboard
 - 12 Troubleshooting
 - 13 Operations
-

What this is

AI Interface is the AI infrastructure layer for the bench. Consumer apps — Excom, DocPulse, WarehouseSuite, BNS, Insightly — import one function and never name a vendor. Swapping Anthropic for Sarvam is a configuration change, not a code change in five repositories.

The three guarantees

GUARANTEE	WHAT IT MEANS IN PRACTICE
Vendor-agnostic	A provider type is a database record holding an adapter path, endpoints, auth header and currency. Any vendor speaking the OpenAI chat-completions format needs zero new Python .
Fully attributed	Every call records which app, module, doctype, document and action caused it — so cost and failures can be traced to a cause, not just a total.
Async by default	Calls enqueue and return an <code>AI Call Log</code> name. Pass <code>sync=True</code> when you need the text inline.

Directory map

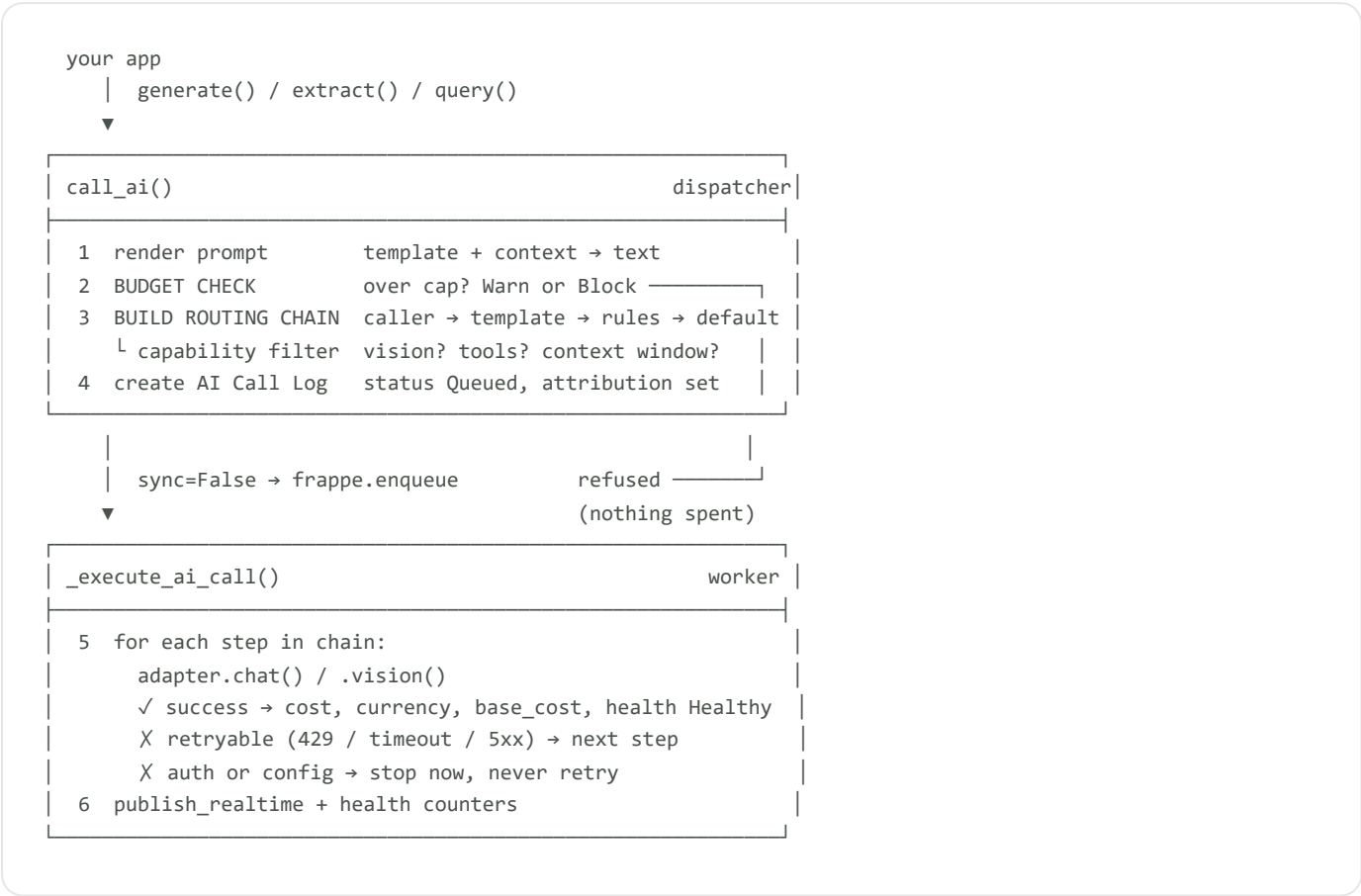
```
ai_interface/
  providers/                                # wire protocols
    base.py                                # BaseProvider ABC, ProviderResponse,
ProviderHTTPError
    openai_compatible.py                    # generic adapter — most vendors need only this
    anthropic_provider.py                  # Anthropic Messages API
    claude_code_provider.py                # shells out to the `claude -p` CLI
    __init__.py                            # resolver: reads AI Provider Type, imports
  adapter_path
  services/                                # the public API consumer apps import
    ai_client.py                           # dispatcher: budget, routing, execute, log, health
    router.py                              # capability routing + fallback chain
    budget.py                              # spend caps
    generator.py vision.py query_engine.py
  api/dashboard.py                         # AI Command Center endpoints
  ai_interface/doctype/                   # the 8 doctypes
  ai_interface/page/ai_command_center/
  patches/v1_0/
```

RULE

Consumer apps import from `services/` only. Never import from `providers/` — that is the layer whose whole purpose is to be swappable.

How a call flows

Six stages. The first three run before anything is queued, so a call that cannot succeed fails immediately and costs nothing.



WHY VALIDATION RUNS FIRST

Sending an image to a text-only model used to fail deep inside a background job with a raw provider 400. It now fails at step 3 with “*Model 'sarvam-105b' on Sarvam does not support images.*”

First-time setup

About five minutes. Steps 1–2 cost nothing; step 4 makes one small billed call.

1 Install and migrate

The migration seeds four **AI Provider Type** records — Anthropic, Claude Code, OpenAI, Sarvam — and creates the **AI Manager** role.

```
bench --site your-site.local install-app  
ai_interface  
bench --site your-site.local migrate
```

2 Set the base currency

AI Settings → *Currency* → *Base Currency*. Every call is converted into this once, at call time, so providers billing in different currencies can be totalled honestly. Pick it before you log real traffic — changing it later does not retro-convert history.

3 Create an AI Provider

New → *AI Provider*. Name it, pick a **Provider Type**, paste the API key, save. Then click **Fetch Models** — free, no tokens. Rates and capabilities fill in automatically for vendors covered by the pricing catalog.

4 Click Test Connection

One real call. Green tells you the credential works for chat; *Fetch Models succeeding is not proof of that*. Provider health flips to **Healthy**.

5 Set the default provider

AI Settings → *Default Provider*. Leave **Default Model** blank and the router picks the cheapest capable model per call — that is the setting that keeps you provider-agnostic.

6 Turn on spend caps

AI Settings → *Spend Caps*. Start with **Warn**. Switch to **Block** once you trust the numbers.

ANTHROPIC KEYS

Use a **workspace-scoped** key. An *organization* key is rejected with `400 – not scoped to a workspace` because it requires an `anthropic-workspace-id` header this app does not send.

Adding any provider

This is the workflow the whole design exists to make cheap. Most vendors are a form entry. Only a genuinely different wire protocol needs code.

Path A — the vendor speaks OpenAI format (no code)

OpenAI, Sarvam, Groq, Together, Mistral, DeepSeek, OpenRouter, Ollama, vLLM and most others. Create an `AI Provider Type` record:

FIELD	VALUE	NOTES
<code>adapter_path</code>	<code>...openai_compatible.OpenAICompatibleProvider</code>	The default. Leave it.
<code>default_base_url</code>	<code>https://api.vendor.com</code>	No trailing slash needed; it is stripped.
<code>chat_path</code>	<code>/v1/chat/completions</code>	Where the vendor accepts chat.
<code>models_path</code>	<code>/v1/models</code>	Blank if the vendor has no listing endpoint — then add models by hand.
<code>auth_header</code>	<code>Authorization</code>	Sarvam uses <code>api-subscription-key</code> .
<code>auth_prefix</code>	<code>Bearer</code>	Blank for raw-key headers. Note the trailing space.
<code>default_currency</code>	<code>USD</code>	What the vendor actually bills in.
<code>unsupported_params</code>	<code>stream_options, service_tier</code>	Request keys the vendor rejects; stripped before sending.

WORKED EXAMPLE — SARVAM

Type Name	Sarvam
Adapter Path	
ai_interface.providers.openai_compatible.OpenAICompatibleProvider	
Default Base URL	https://api.sarvam.ai
Chat Path	/v1/chat/completions
Models Path	/v2/models
Auth Header	api-subscription-key
Auth Prefix	(empty)
Default Currency	INR
Unsupported	stream_options, max_completion_tokens, service_tier

Then: New AI Provider → type *Sarvam* → paste `sk_...` → Fetch Models → type the ₹ rates → Test Connection. **No Python was written.**

Path B — a different wire protocol (one file)

1

Write an adapter

Subclass `BaseProvider` in `providers/`, implementing `chat()`, `vision()` and `fetch_models()`. Read vendor config from `self.config`.

2

Point a record at it

Create an `AI Provider Type` whose `adapter_path` is the dotted path to your class. It is validated on save, so a typo fails then — not during a call at 2am.

EXTENSIBLE BY OTHER APPS

Because registration is a record rather than a line in

`providers/__init__.py`, another app can ship its own adapter and register it with a fixture, without ever editing this one.

Using it from your app

Four entry points. All share the same keyword arguments; all return an `AI Call` `Log` name by default, or the response text when `sync=True` .

generate() — text

```
from ai_interface.services.generator import generate

log_name = generate(
    prompt="Summarise this thread in two sentences.",
    calling_app="excom",
    reference_doctype="Excom Thread",
    reference_name=thread.name,
    action="summarize_thread",
)

# or with a stored template
text = generate(
    template="email_draft",
    context={"customer": "Acme"},
    sync=True,
    calling_app="excom",
)
```

extract() — images and PDFs

```
from ai_interface.services.vision import extract

log_name = extract(
    file_url="/private/files/invoice.pdf",
    prompt="Extract supplier, date, line items and grand total as JSON.",
    calling_app="docpulse",
    reference_doctype="Purchase Invoice",
    reference_name=pi.name,
    action="extract_invoice",
)
```

PDFs are rasterised page by page up to `max_pdf_pages` . Vision is implied — you do not pass `needs` .

query() — natural language over ERPNext data

```
from ai_interface.services.query_engine import query

answer = query("How many customers ordered this month?",
               calling_app="insightly", sync=True)
```

COSTS TWO CALLS

One `query_generation` and one `answer_synthesis` , both logged and both attributed to your `calling_app` . Budget accordingly.

Shared arguments

ARGUMENT	TYPE	PURPOSE
<code>calling_app</code>	<code>str</code>	Which app caused this. The top level of every cost breakdown — always pass it.
<code>reference_doctype</code>	<code>str</code>	Doctype this call serves. The module is derived from it automatically.
<code>reference_name</code>	<code>str</code>	The specific document, so a cost traces to one record.
<code>action</code>	<code>str</code>	What the call does, e.g. <code>extract_invoice</code> . Groups identical work across documents.
<code>needs</code>	<code>list[str]</code>	<code>["vision"]</code> , <code>["tools"]</code> . State capabilities, never a model name.
<code>sync</code>	<code>bool</code>	Inline execution returning text. Default <code>False</code> → enqueued, returns a log name.
<code>provider / model</code>	<code>str</code>	Hard override. Wins over every rule — use sparingly, it is what makes code vendor-locked.
<code>template</code>	<code>str</code>	An <code>AI Prompt Template</code> name; combine with <code>context</code> .
<code>max_tokens</code>	<code>int</code>	Ceiling, not a charge. Raise it for reasoning models.

Collecting an async result

```
# server side
log_name = generate(prompt=..., calling_app="excom")

# client side – the worker publishes on completion
frappe.realtime.on("ai_call_complete", (d) => {
    if (d.log === log_name) frappe.db.get_value("AI Call Log", d.log,
"output_text")
});
frappe.realtime.on("ai_call_failed", (d) => { /* d.error */ });
```

Doctype reference

Eight doctypes. Three are child tables, one is a Single. Every field below exists in the current schema.

AI Provider Type

AUTONAME: TYPE_NAME

The registry, as data. One record per vendor protocol. This is what replaced a hardcoded Python dictionary — and why a new vendor is configuration.

FIELD	TYPE	WHAT IT IS FOR
<code>type_name</code>	Data · reqd	Vendor name shown in the AI Provider form.
<code>enabled</code>	Check · 1	Disabling refuses every call routed through this type.
<code>adapter_path</code>	Data · reqd	Dotted path to a <code>BaseProvider</code> subclass. Imported and validated on save.
<code>default_base_url</code>	Data	Vendor API root. An AI Provider may override it.
<code>chat_path</code>	Data	Chat completions path. Default <code>/v1/chat/completions</code> .
<code>models_path</code>	Data	Model listing path. Blank ⇒ no discovery, enter models by hand.
<code>no_credential</code>	Check · 0	Adapter authenticates itself (a local CLI). Hides the key fields.
<code>auth_header</code>	Data	Header carrying the credential. Default <code>Authorization</code> .
<code>auth_prefix</code>	Data	Prepended to the key. Default <code>Bearer</code> ; blank for raw-key headers.
<code>default_currency</code>	Link Currency	What the vendor bills in. Seeds the provider's currency.
<code>pricing_source_url</code>	Data	JSON catalog used to prefill rates. Clear it to disable all outbound lookups.
<code>catalog_prefix</code>	Data	Prefix tried when matching ids in that catalog.
<code>unsupported_params</code>	Small Text	Request keys this vendor rejects; stripped before sending.

AI Provider

AUTONAME: PROVIDER_NAME

One configured account. You may have several of the same type — two Anthropic workspaces, or an Exotel-style split by department.

FIELD	TYPE	WHAT IT IS FOR
provider_name	Data · reqd	Also the record name; used in routing rules and call logs.
provider_type	Link · reqd	Which wire protocol and endpoints to use.
enabled	Check · 1	Disabled providers are skipped by the router.
auth_type	Select	API Key or Auth Token (OAuth).
api_key	Password	Encrypted at rest. sk-ant-api03-... for Anthropic.
auth_token	Password	OAuth token from claude setup-token .
api_base_url	Data	Per-account override of the type's base URL.
currency	Link Currency	What this account is billed in. Defaults from the type.
exchange_rate	Float	Manual rate to base currency. 0 ⇒ use the ERPNext Currency Exchange rate at call time.
models	Table	The catalog. See <i>AI Provider Model</i> .
health_status	Select · ro	Unknown / Healthy / Degraded / Down . Written by every call.
last_success	Datetime · ro	When it last answered.
last_failure	Datetime · ro	When it last did not.
consecutive_failures	Int · ro	Reset to 0 on success. Crossing failure_threshold ⇒ Down.
last_error	Small Text · ro	Classified type plus the vendor's own message.

BUTTONS

Fetch Models

Free. Discovers ids, merges — never wipes hand-entered rates.

Test Connection One small billed call on the cheapest enabled model. Sets health.

AI Provider Model

CHILD OF AI PROVIDER

The model catalog and price list. Discovery fills the ids; rates and capabilities come from the catalog or from you.

FIELD	TYPE	WHAT IT IS FOR
model_id	Data · reqd	Exactly what the vendor's API expects.
label	Data	Human-readable name from discovery.
enabled	Check · 1	Cleared automatically when a vendor stops returning it. Not deleted — old logs still cost against it.
supports_vision	Check	Gate for image calls. The router will not send images to a model without it.
supports_tools	Check	Gate for <code>needs=["tools"]</code> .
context_window	Int	Max input tokens. Oversized prompts are refused before queueing.
priority	Int · 0	Lower is preferred when auto-picking. Ties break on cost.
cost_per_input_token	Float	In the <i>provider's</i> currency. Divide the per-million price by 1,000,000.
cost_per_output_token	Float	Same.
pricing_source	Select · ro	<code>Manual</code> (never overwritten) · <code>Catalog</code> · <code>Unpriced</code> .

READING A PRICE

Sarvam lists `gemma4` at ₹36.6 per 1M input tokens. Enter `0.0000366`. Saving stamps it `Manual`, and no future Fetch Models will touch it.

AI Settings

SINGLE

Bench-wide policy: defaults, routing, currency, caps, health thresholds.

FIELD	TYPE	WHAT IT IS FOR
default_provider	Link · reqd	Runs calls when nothing more specific matches.
default_model	Data · reqd	Pin a model — or leave it to let the router pick the cheapest capable one.
api_call_timeout	Int · 120	Seconds before giving up. Raise to 300 for long PDF extractions.
max_output_tokens	Int · 4096	Default ceiling per call. A ceiling, not a charge.
max_pdf_pages	Int · 10	Pages rasterised by <code>extract()</code> . Each adds roughly 1,500 input tokens.
enable_logging	Check · 1	Controls <i>payload retention</i> — prompt and response bodies. The log row is always written.
base_currency	Link · reqd	The one currency all spend is normalised into.
routing_rules	Table	See <i>AI Routing Rule</i> .
enable_fallback	Check · 1	Try the next rule on retryable failures only.
enable_budget	Check · 0	Master switch for spend caps.
daily_budget	Currency · 0	0 ⇒ no daily cap.
monthly_budget	Currency · 0	0 ⇒ no monthly cap. Calendar month.
budget_action	Select · Warn	<code>Warn</code> notifies and lets calls through; <code>Block</code> refuses them.
budget_alert_threshold	Int · 80	Percent of a cap that triggers a notification.
app_budgets	Table	See <i>AI App Budget</i> .
failure_threshold	Int · 3	Consecutive failures before a provider reads as Down.

AI Routing Rule CHILD OF AI SETTINGS

Which provider serves which work. Matched most-specific-first, then by priority — so later rules become the fallback chain.

FIELD	TYPE	WHAT IT IS FOR
<code>enabled</code>	Check · 1	Turn a rule off without deleting it.
<code>function_type</code>	Select	<code>Generation</code> / <code>Vision</code> / <code>Query</code> . Blank matches any.
<code>calling_app</code>	Data	Restrict to one app. Blank matches any.
<code>provider</code>	Link · reqd	Which provider serves the match.
<code>model</code>	Data	Leave blank to auto-pick the cheapest capable model.
<code>priority</code>	Int · 0	Lower runs first; the rest become fallbacks.

AI App Budget CHILD OF AI SETTINGS

Per-app caps, so a runaway job cannot drain the budget other apps need.

FIELD	TYPE	WHAT IT IS FOR
<code>calling_app</code>	Data · reqd	Must match the <code>calling_app</code> your code passes.
<code>daily_budget</code>	Currency	0 ⇒ no daily cap for this app.
<code>monthly_budget</code>	Currency	0 ⇒ no monthly cap for this app.
<code>budget_action</code>	Select	Blank falls back to the global setting.

AI Prompt Template

AUTONAME: TEMPLATE_NAME

Versioned Jinja prompts, so wording changes without a deploy.

FIELD	TYPE	WHAT IT IS FOR
template_name	Data · reqd	What you pass as <code>template=</code> .
category	Select · reqd	<code>Generation / Vision / Query / System</code> .
body	Code · reqd	Jinja, rendered against <code>context</code> .
variables	Small Text	JSON list of expected names, e.g. <code>["customer_name"]</code> .
provider_override	Link	Force a provider for this template. Beats routing rules.
model_override	Data	Force a model for this template.

AI Call Log

AUTONAME: HASH

One row per call — the audit trail, the async delivery channel, and the sole data source for the dashboard. Every field is read-only.

IDENTITY & ROUTING

FIELD	TYPE	WHAT IT IS FOR
status	Select	Queued → Running → Completed / Failed .
function_type	Select	Generation / Vision / Query .
provider	Link	Who <i>actually</i> served it — updated if the router fell back.
model	Data	The model that answered, as the vendor reported it.
is_sync	Check	Inline or enqueued.
attempts	Int · 1	More than 1 ⇒ the router fell back after a retryable failure.
prompt_template	Link	Template used, if any.

ATTRIBUTION — WHAT MAKES COST EXPLAINABLE

calling_app	Data	Top level of every breakdown.
module	Link Module Def	Derived from the reference doctype automatically.
reference_doctype	Link DocType	What kind of record this served.
reference_name	Dynamic Link	Which specific record.
action	Data	The operation, e.g. <code>extract_invoice</code> .
user	Link User	Who triggered it.

PAYLOAD, COST AND OUTCOME

input_text / output_text	Long Text	Retained only while <code>enable_logging</code> is on.

error_type	Select	Auth · Rate Limit · Timeout · Invalid Response · Provider Error · Config Error · File Error · Unknown . Derived from HTTP status, not error text.
error_message	Long Text	Full trail. Kept even on success if an earlier provider failed first.
input_tokens / output_tokens	Int	As reported by the vendor.
currency	Link	What the serving provider bills in.
cost	Currency	Billed amount, in that currency.
exchange_rate	Float	Rate at call time, snapshotted so later changes never rewrite history.
base_cost	Currency	What every dashboard total sums.
base_currency	Link	The base at the time of the call.
latency_ms	Int	Wall-clock for the attempt that answered.

INDEXES

Declared in `on_doctype_update()`: `(status, creation)`, `(calling_app, creation)`, `(module, creation)`, `(reference_doctype)`, `(provider, model)`.

How routing decides

Two passes: pick candidate providers, then filter by what the call actually needs.
What comes out is a chain — first entry serves, the rest are fallbacks.

Pass 1 — candidates, in order

- 1

Caller override

`provider=` / `model=` passed in code. Wins outright.
- 2

Template override

`provider_override` on the AI Prompt Template.
- 3

Routing rules

Most-specific first: app + function beats function alone beats catch-all.
Priority orders equally specific rules.
- 4

AI Settings default

The last resort, and the tail of the fallback chain.

WHY SPECIFICITY BEATS PRIORITY

A rule written for one app must never be shadowed by a lower-priority catch-all that happens to sort first. Specificity is compared before priority, always.

Pass 2 — capability filter

REQUIREMENT	WHERE IT COMES FROM	CHECKED AGAINST
vision	<code>needs=["vision"]</code> , or implied by <code>images</code>	<code>supports_vision</code>
tools	<code>needs=["tools"]</code>	<code>supports_tools</code>
min_context	Prompt length ÷ 4 characters per token	<code>context_window</code>

A model named explicitly that cannot meet the requirement **throws**. A model being auto-picked simply excludes it, then sorts survivors by `priority` , then by combined token cost — so the default is the cheapest model that can do the job.

Fallback

ERROR CLASS	BEHAVIOUR	WHY
Rate Limit	Try next provider	Transient; another vendor is not throttled.
Timeout	Try next provider	Transient.
Provider Error	Try next provider	A 5xx is the vendor's problem, not the request's.
Auth	Stop immediately	A bad key fails the same way every time — retrying just costs money and time.
Config Error	Stop immediately	Misconfiguration is not fixed by repetition.

Set `enable_fallback = 0` to make every chain a single entry.

Cost & currency

Providers do not all bill in the same currency. Sarvam prices in ₹, Anthropic in \$. Summing those raw would produce a meaningless number, so cost is stored twice.

FIELD	HOLDS	USED FOR
cost + currency	What the provider actually billed, in its own currency	Reconciling against a vendor invoice
base_cost + base_currency	The same amount converted once, at call time	Every dashboard total
exchange_rate	The rate used, snapshotted	History that a later rate change cannot rewrite

Rate resolution, in order

- 1

Same currency

Rate is 1.0.
- 2

Manual override

exchange_rate on the AI Provider, if greater than zero.
- 3

ERPNext Currency Exchange

Looked up live for the pair.
- 4

Nothing usable ⇒ 0

Never a silent 1.0.

WHY AN UNRESOLVABLE RATE STORES ZERO

A silent 1.0 would treat ₹100 as \$100 — the exact error this design exists to prevent. Instead the call is left visibly unconverted and the dashboard raises “*N calls are missing from every cost total*”. Fix it by setting the rate on the provider or adding a Currency Exchange record.

Display currency

The picker in the dashboard filter bar converts `base_cost` at read time. Switching between ₹ and \$ is a lens over the same history, never a rewrite of it. An unavailable rate falls back to base rather than mis-scaling the page.

Spend caps

Checked before a call is enqueued, so a blocked call costs nothing. Measured against `base_cost` , so mixed-currency providers sit under one budget.

SCOPE	PERIODS	CONFIGURED IN
Global	Daily, monthly (calendar)	AI Settings → Spend Caps
Per app	Daily, monthly	AI Settings → Per-App Caps

- Warn Notifies System Managers and lets the call through.
- Block Refuses the call before it costs anything. If any applicable rule says Block, Block wins.
- Alerts Fire once spend crosses `budget_alert_threshold` , throttled to one per scope per day.

START WITH WARN

A hard block that silently breaks five apps' AI features is a worse first experience than an alert. Move to Block once the numbers are trusted.

Provider health

When calls start failing, the first question is always *which provider*. These counters answer it without reading tracebacks.

STATUS	MEANING
Unknown	Never called. Click Test Connection.
Healthy	Last call succeeded; failure counter is 0.
Degraded	Failing, but below <code>failure_threshold</code> .
Down	Consecutive failures reached the threshold.

Updated by every call on both the success and failure paths, written with `db_set` so health bookkeeping can never itself fail a call. Surfaced in the AI Provider list, on the form header, and as the dashboard health strip.

TEST CONNECTION VS FETCH MODELS

Fetch Models proves nothing about chat. A credential can list models happily and still be rejected for completions, or belong to an account with no balance. Test Connection sends one real, minimal call — the only proof that matters.

The dashboard

`/app/ai-command-center` — visible to System Manager and AI Manager. Every endpoint returns pre-aggregated rows; nothing fetches log documents, so it stays O(groups) rather than O(calls).

PANEL	ANSWERS
Health strip	Which providers are answering right now.
Budget strip	Spend against caps, with an over-budget state.
KPIs	Spend, calls, success rate, p95 latency, deltas vs the previous window.
Insights	Cost spikes, prompt bloat, repeated prompts, failure clusters, unconverted spend, fallback activity.
Spend over time	Stacked daily cost, regroupable by app, module, function, model or provider.
Attribution	Drill-down: app → module → doctype → action.
Reliability	Daily success rate and per-model cost per call.
Failures	By error type, a module × day heatmap, recent rows with Retry.

ENDPOINTS — `AI_INTERFACE.API.DASHBOARD`

`get_summary` · `get_timeseries` · `get_attribution` · `get_reliability` ·
`get_failures` · `get_insights` · `get_provider_health` ·
`get_budget_status` · `get_filter_options` · `retry_call`

Troubleshooting

Every entry here is a failure actually hit while building and testing this app.

SYMPTOM	CAUSE & FIX
400 – This API key is not scoped to a workspace	An Anthropic <i>organization</i> key. Use a workspace key instead.
This endpoint is currently in beta and not available	Sarvam <code>/v2/chat/completions</code> is gated per account. Set <code>chat_path</code> to <code>/v1/chat/completions</code> . Discovery on <code>/v2/models</code> still works.
“used its entire max_tokens budget on reasoning”	A reasoning model such as <code>sarvam-105b</code> fills <code>reasoning_content</code> before <code>content</code> . Raise <code>max_tokens</code> — it is a ceiling, not a charge.
Cost logs as 0 on completed calls	The model row is <code>Unpriced</code> . Enter the rates on the AI Provider; they stamp as <code>Manual</code> and survive re-fetch.
“N calls are missing from every cost total”	No exchange rate into the base currency. Set <code>exchange_rate</code> on the provider or add a Currency Exchange record.
“has no enabled model that supports images”	Working as designed — refused before queueing. Enable a vision model, or route Vision work to a provider that has one.
Enter the API Key for ... before using it	The provider was saved without a credential.
“Global monthly AI budget exhausted”	A cap with <code>Block</code> . Raise it, switch to Warn, or wait for the next period.
Page 404 after install	<code>bench --site ... clear-cache</code> , then hard-refresh the browser.
Rates vanished after Fetch Models	Should be impossible — merge preserves <code>Manual</code> rows. If it happens, that is a bug worth reporting.

Operations

Everyday commands

```
bench --site <site> migrate           # after any doctype change
bench --site <site> clear-cache        # after editing page JS or CSS
bench --site <site> console            # interactive, with frappe loaded

cd apps/ai-interface
ruff check ai_interface/ && ruff format ai_interface/
```

Patches

PATCH	DOES
create_ai_manager_role	Creates the read-only AI Manager role.
create_default_provider_types	Seeds Anthropic, Claude Code, OpenAI and Sarvam. Skips any that exist.
backfill_base_cost	Copies cost into base_cost at rate 1 for pre-currency history, and sets the base to USD.
sarvam_chat_path_v1	Moves seeded Sarvam records off the beta-gated /v2 , leaving deliberate overrides alone.

Roles

System Manager	Full access. Required for Fetch Models and Test Connection.
AI Manager	Read, report and export on AI Call Log; can open the dashboard. No configuration rights.

Clearing test data

```
bench --site <site> mariadb
DELETE FROM `tabAI Call Log`;
```

CHECK BEFORE YOU RUN IT

This removes real history as well as test rows. There is no undo, and the dashboard has no other data source.

Known gaps

- **No automated test suite.** Behaviour has been verified by hand and against live accounts, but nothing guards it against the next change.
- **Test Connection calls are not written to AI Call Log**, so that (small) spend is untracked.
- **Scheduled health checks are not implemented** — health updates only when real calls run.
- **Sarvam's non-chat APIs** (TTS, STT, translate, transliterate) do not fit the chat/vision contract and would need their own function type.

Reference for `ai_interface` on Frappe/ERPNext v15 · branch `somil-dev` . Field lists generated from the current doctype schema.